COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	16/10/2026	**DATE SUBMITTED:**	20/10/2026
NAME:	Lopez, Andrei Dion Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

Screenshot

```
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str){
    std::cout<<"Printing integer: "<<n<<std::endl;
    std::cout<<"Printing string: "<<str<<std::endl;
}

int main(){

        std::thread t1(print, 10, "T.I.P.");
        t1.join();

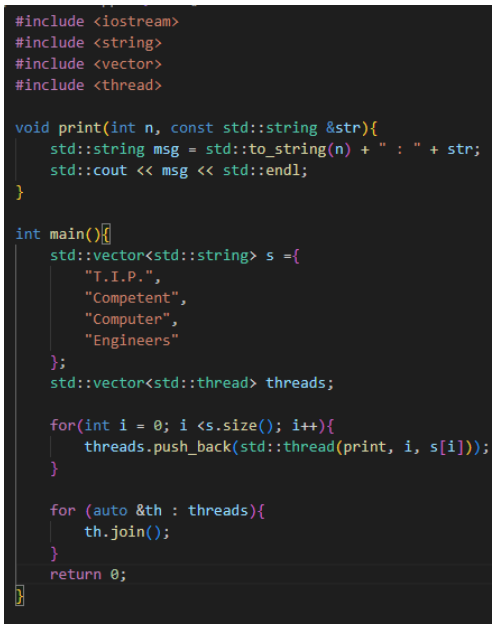
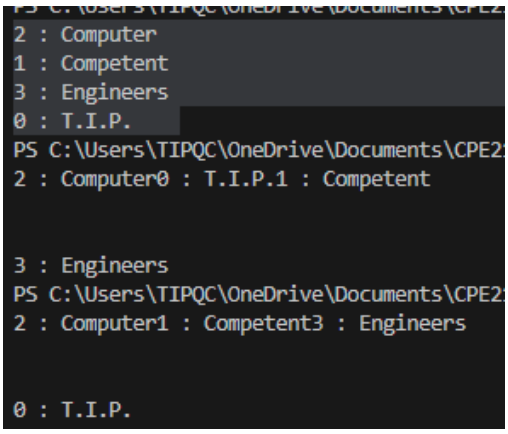
    return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

In this program, we first have the thread library to allow the use of thread mainly for the function pointer. After creating the function print, we initialize the main and test both the thread and the class which essentially looks similar to a simple cin and cout except they are predefined within the call of thread itself.

Table 13-1. Simple One-Threaded Example

Screenshot	 
Analysis	The program has a for loop that creates the threads in order but it does not specifically mean that it will run first. From the screenshot of running the program thrice, the order changes from each run, leading to the different outputs despite no changes in the syntax itself.

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <thread>

void print(int n, const std::string &str){
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main(){
    std::vector<std::string> s ={
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for(int i = 0; i <s.size(); i++){
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads){
        th.join();
    }
    return 0;
}
```

```
2 : Computer
1 : Competent
3 : Engineers
0 : T.I.P.
PS C:\Users\TIPQC\OneDrive\Documents\CPE2
2 : Computer0 : T.I.P.1 : Competent


3 : Engineers
PS C:\Users\TIPQC\OneDrive\Documents\CPE2
2 : Computer1 : Competent3 : Engineers


0 : T.I.P.
```

Table 13-2.Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PART A	
LOPEZ	**1. Write a definition of multithreading and its advantages/disadvantages.** **- Multithreading is the process of letting multiple threads share the responsibility of running a computer. Ranging from memory, CPU power, and responsiveness. Although it carries many benefits, it does include the risk of deadlock or a race between two threads, or even difficulty when debugging threads.** **2. Rationalize the use of multithreading by providing at least 3 use-cases.** **- Multi-threading can be beneficial especially when using several different applications at once, through web servers, or even with video games.** **3. Differentiate between parallelism and concurrency** **- Parallelism tackles conducting many tasks all at once in a machine, while Concurrency focuses similarly, but instead of all at once, it works by overlapping one another.**
REALUBIT	**1. Write a definition of multithreading and its advantages/disadvantages.** **- Multithreading lets one process run multiple threads that share the memory, improving responsiveness, CPU use, and scalability, but it poses the risks of race conditions and deadlocks, and it becomes harder to debug because of the timing.** **2. Rationalize the use of multithreading by providing at least 3 use cases.** **- Some uses of multithreading are web servers, GUI apps, and games that must run without blocking one another.** **3. Differentiate between parallelism and concurrency.** **- Concurrency means managing many tasks at once, which leads to overlapping, while parallelism is similar to having another machine, wherein it is distributed evenly.**

PART B

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

int globalValue = 0;
std::mutex mtx;

void add(int value, int delayMs) {

    std::this_thread::sleep_for(std::chrono::milliseconds(delayMs));
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += value;
}

void display(const std::string &label) {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << label << globalValue << std::endl;
}

int main() {
    std::thread t1(add, 10, 100);
    std::thread t2(add, 20, 300);
    std::thread t3(add, 30, 500);

    display("Before any join : ");
    t1.join();
    display("After T1.join() : ");
    t2.join();
    display("After T2.join() : ");
    t3.join();
    display("After T3.join() : ");

    return 0;
}
```

```
Before any join : 0
After T1.join() : 10
After T2.join() : 30
After T3.join() : 60
```

ANALYSIS	
LOPEZ	The program utilized a mutex to keep the three threads from changing the global variable at the same time, which prevents two threads from reading the same old value. We then proceed to join the threads one at a time. After the first join, the value of 10 from the first thread is guaranteed to be included, and after the second join, the values of 10 and 20 are both included, which brings the total to at least 30. Afterwards, the sleep_for function is added to each thread so that the progression of the values can be seen in the output. Without it, the threads would finish too quickly and the output would simply appear all at once.
REALUBIT	By using the mutex, we are able to separate the threads properly, preventing the possibility of two threads reading the same old value. Apart from that, the first join of 10, then the 2nd join of 10 and 20 are guaranteed, thus leading to the value of 30. Lastly, we have the "sleep_for" so that we are able to easily see the output of the implementation, as without it, it would be too fast for us to see and the output would just appear right away.

PART C

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <algorithm>

int minSteps(int n) {
    std::vector<int> dp(n + 1, 0);
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + 1;                          // subtract 1
        if (i % 2 == 0) dp[i] = std::min(dp[i], dp[i / 2] + 1);  // divide by 2
        if (i % 3 == 0) dp[i] = std::min(dp[i], dp[i / 3] + 1);  // divide by 3
    }
    return dp[n];
}

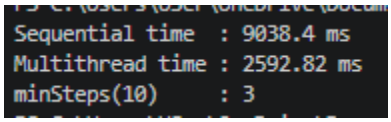
void worker(int id, std::vector<int> &results, int N) {
    for (int n = id; n <= N; n += 4) {
        results[n] = minSteps(n);
    }
}

int main() {
    const int N = 30000;
    std::vector<int> results(N + 1);

    // Sequential
    auto start = std::chrono::high_resolution_clock::now();
    for (int n = 1; n <= N; n++) results[n] = minSteps(n);
    auto end = std::chrono::high_resolution_clock::now();
    double seqTime = std::chrono::duration<double, std::milli>(end - start).count();

    // Multithreaded
    start = std::chrono::high_resolution_clock::now();
    std::thread t1(worker, 1, std::ref(results), N);
    std::thread t2(worker, 2, std::ref(results), N);
    std::thread t3(worker, 3, std::ref(results), N);
    std::thread t4(worker, 4, std::ref(results), N);
    t1.join();
    t2.join();
    t3.join();
    t4.join();
    end = std::chrono::high_resolution_clock::now();
    double parTime = std::chrono::duration<double, std::milli>(end - start).count();

    std::cout << "Sequential time  : " << seqTime << " ms\n";
    std::cout << "Multithread time : " << parTime << " ms\n";
    std::cout << "minSteps(10)     : " << results[10] << "\n";
    return 0;
}
```



ANALYSIS	
LOPEZ	The program itself applied the bottom-up approach of dynamic programming and split the work across 4 threads. Each value of n is independent, and every thread only writes to its own indices of the results vector, so no race condition can happen. By dividing the work this way, the total time becomes shorter compared to the sequential loop. This is possible because the threads run on different cores at the same time. However, the speedup stays below 4x because of the time spent creating the threads and the scheduling overhead. Resulting in the threaded version being slower for small values of N, since the overhead becomes bigger than the work that was split.
REALUBIT	Each n is independent, and each thread writes to different indices of results, so there is no race. By splitting the work across 4 threads, we are able to reduce the total time compared to the sequential loop. The speedup will be below 4x because of the creation of threads and scheduling overhead. Lastly, for small values of N, the threaded version can be slower because the overhead is bigger than the work being split.

CONCLUSION	
LOPEZ	In this activity we discovered two more concepts for handling tasks, which are parallelism and multithreading. Parallelism splits a task into smaller subtasks that run at the same time, while multithreading uses multiple threads within a single process. During the procedure, we programmed examples that print strings with threads and observed that the output order changed on every run. For the supplementary activity, we used the mutex library in Part B to protect the global variable and prevent overlaps that would break the results. We also applied multithreading to the bottom-up approach from a previous activity and compared it with the sequential version. Despite our findings, there was some difficulty in analyzing the outputs, mainly in explaining why the results changed between runs. Overall, I would like to think we did decently in this activity, but I still need to improve my code analysis.
REALUBIT	In this hands-on activity, we discussed about 2 more algorithms which is parallel and multithreading. In summary, parallel processing uses a divide-and-conquer approach by splitting tasks into smaller subtasks, while multithreading uses a single process. During the procedure, we programmed the two different algorithms in printing strings and showed how fast each would go and work. During the supplementary, we used a new library for part b to allow the protection of data and prevent overlaps that would break the sequence, along with using the multithreading and sequential algorithm in one of the past activities, such as the bottom-up approach with some trade-offs of not scaling with the CPU anymore. Overall, my

	performance for this activity was decent, but still with room for improvement in my code analysis.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The use of AI was explicitly for errors and new concepts, such as the mutex library, and its use of it would help with the output of the program**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.